

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn – Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B

Nhóm bài tập lớn: 10

Sinh viên thực hiện: Nguyễn Minh Toàn

Mã sinh viên: B23DCCN832

Báo cáo kiến trúc và giao diện hệ thống

Phần cá nhân: *Phân hệ tầng giao vận UDP (Transport)*

Hà Nội – 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp mã nguồn	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu – cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN – Phân hệ tầng giao vận UDP (Transport).	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Kiến trúc thư viện Transport	13
3.2. Lựa chọn UDP và mô hình lập trình socket	13
3.3. Định dạng gói tin GSP	16
3.4. Bắt tay không lưu trạng thái	17
3.5. Máy trạng thái kết nối	18
3.6. Cơ chế ack bằng bitfield và gửi lại	18
3.7. Kênh, phân mảnh và kiểm soát nghẽn	20
3.8. Giao diện lập trình và giao diện chẩn đoán	21
4. Luồng xử lý và giải pháp đã ứng dụng	21
4.1. Luồng thiết lập kết nối UDP có xác thực	22
4.2. Luồng gửi và nhận một bản tin	24
4.3. Luồng phát hiện mất gói và gửi lại	26
4.4. Bốn kênh truyền và luồng phân loại khi nhận	27
4.5. Luồng cắt và ghép mảnh bản tin lớn	27
4.6. Luồng duy trì kết nối, kiểm soát nghẽn và ngắt kết nối	29
4.7. Bộ giả lập mạng, ghi phát lại gói và đo hiệu năng	30
4.8. Kiểm thử phân hệ	31

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

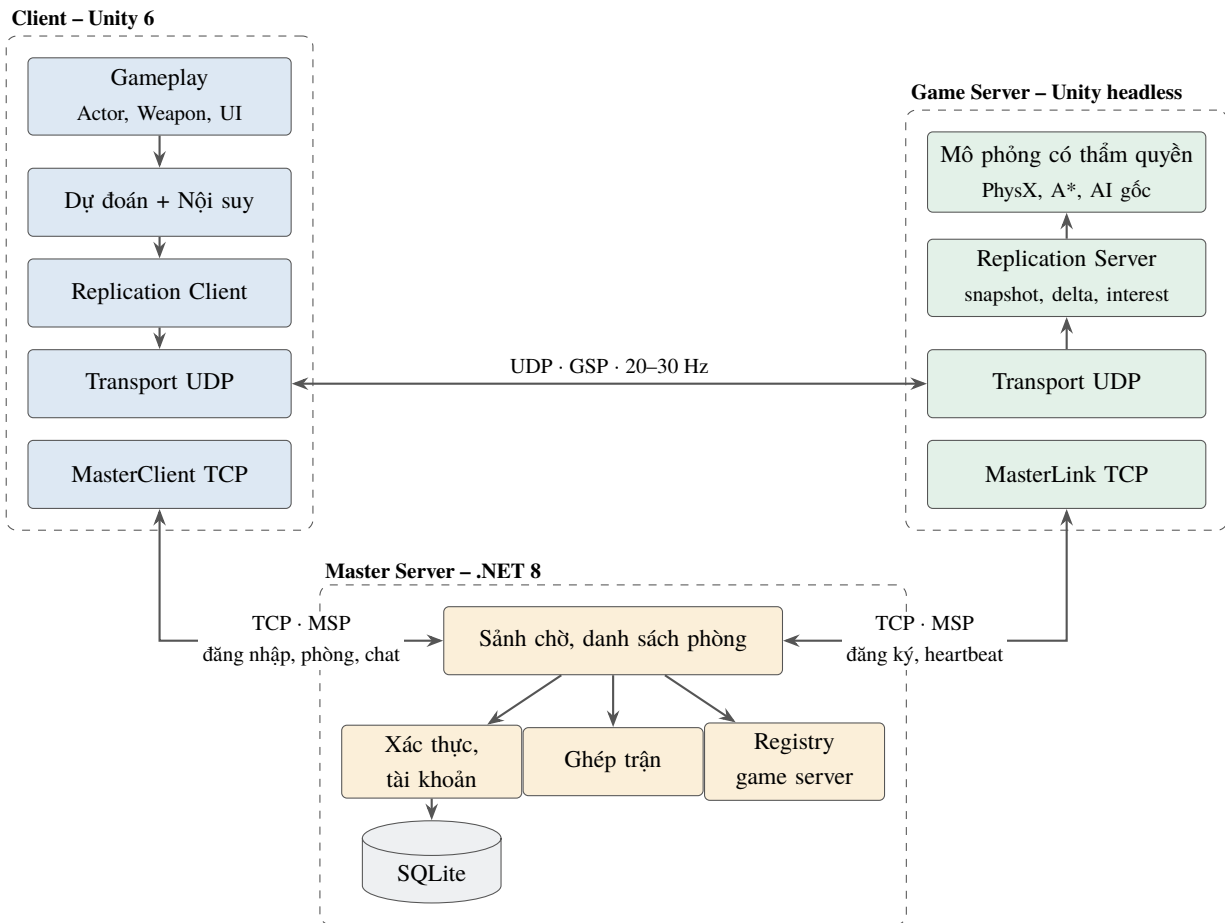
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **nhiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP** thô qua `System.Net.Sockets` — không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ sảnh chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (UNITY_SERVER)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

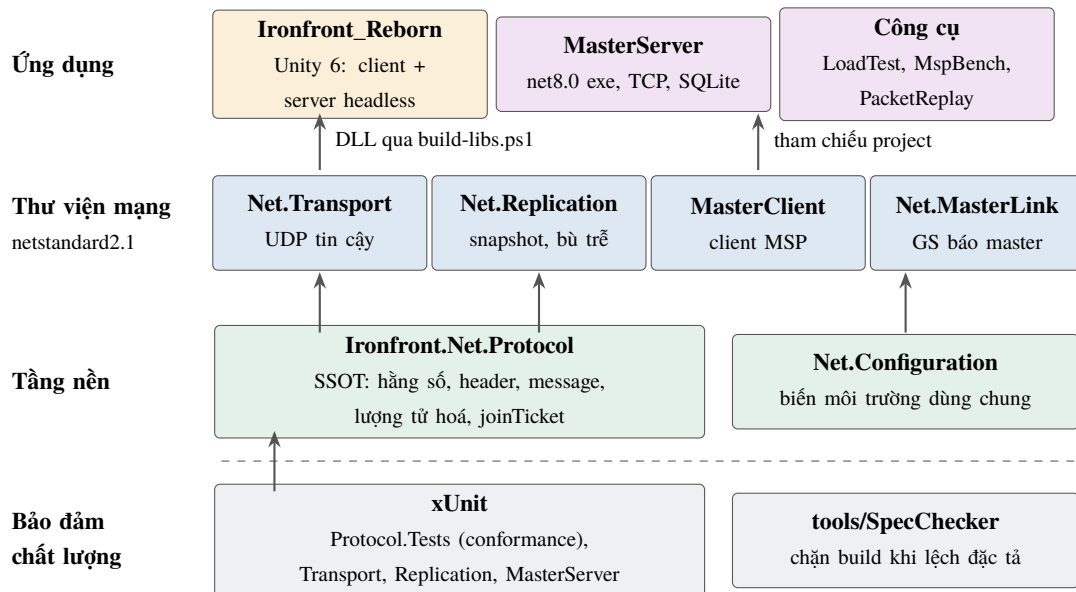
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP – Master Server	UDP – Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20–60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40–200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP — một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp mã nguồn

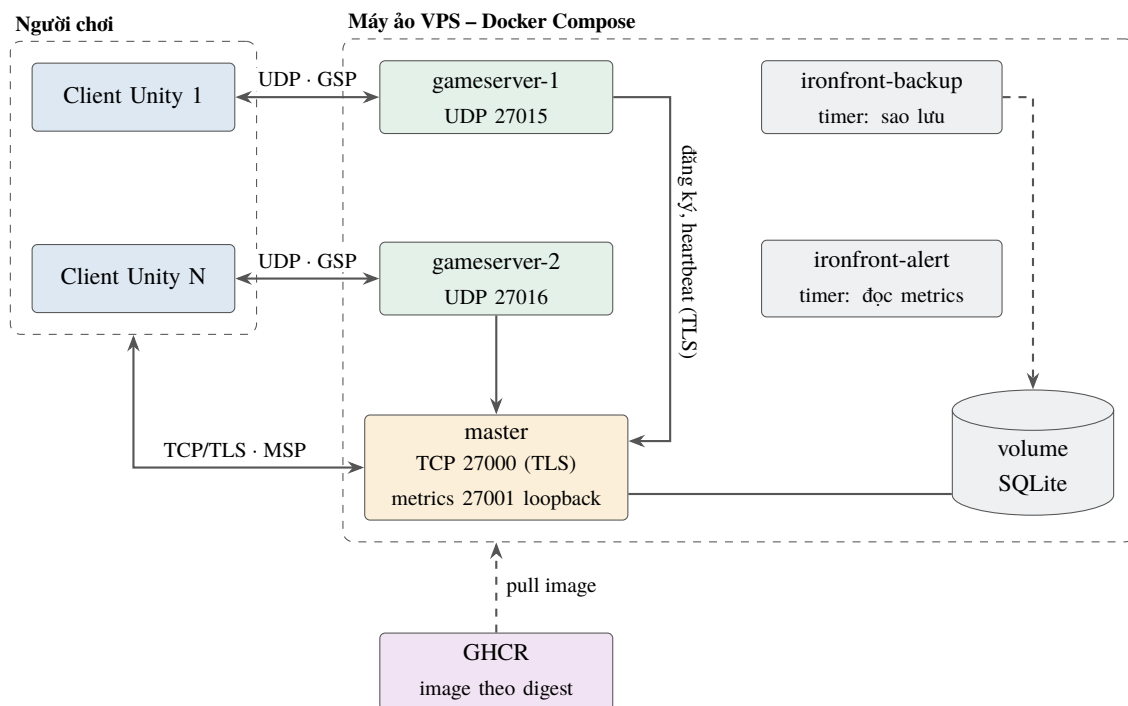
Mã nguồn chia thành các thư viện .NET thuần (netstandard2.1) **không bao giờ tham chiếu** UnityEngine; dự án Unity dùng chúng dưới dạng DLL do tools/build-lib.ps1 sinh vào Assets/Plugins. Nhờ vậy logic mạng được kiểm thử bằng xUnit trong vài giây, không cần mở Unity Editor.



Hình 1.2. Phân lớp thư viện và quan hệ phụ thuộc giữa các project trong solution

Project	Loại	Dòng mã	Vai trò
Ironfront.Net.Protocol	netstandard2.1	5 300	Nguồn sự thật duy nhất của định dạng dây
Ironfront.Net.Transport	netstandard2.1	4 100	Độ tin cậy trên UDP, kênh, phân mảnh, nghẽn, giả lập mạng
Ironfront.Net.Replication	netstandard2.1	18 500	Snapshot, delta, interest, tick server, bù trễ, trận đấu
Ironfront.MasterServer	net8.0 exe	4 300	Framing, dispatch, xác thực, sánh, ghép trận, TLS, metrics
MasterClient, MasterLink	netstandard2.1	1 250	API cho client Unity và liên kết game server – master
Assets/Scripts/Net	Unity C#	14 600	Lớp keo Unity: bootstrap, dự đoán, hiển thị, giao diện

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được kéo từ GHCR theo digest cố định; bí mật (IRONFRONT_SHARED_SECRET, chứng chỉ TLS) lấy từ tệp .env không commit. Sao lưu và cảnh báo chạy bằng systemd timer ngoài container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình Terraform (tạo VM, cloud-init) và fly.io làm phương án thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi mã nguồn chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Assets/Scripts/Net/Client, Net/Input
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Ironfront.Net.Transport
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Ironfront.Net.Replication, Net/Server
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Ironfront.MasterServer, MasterClient, infra

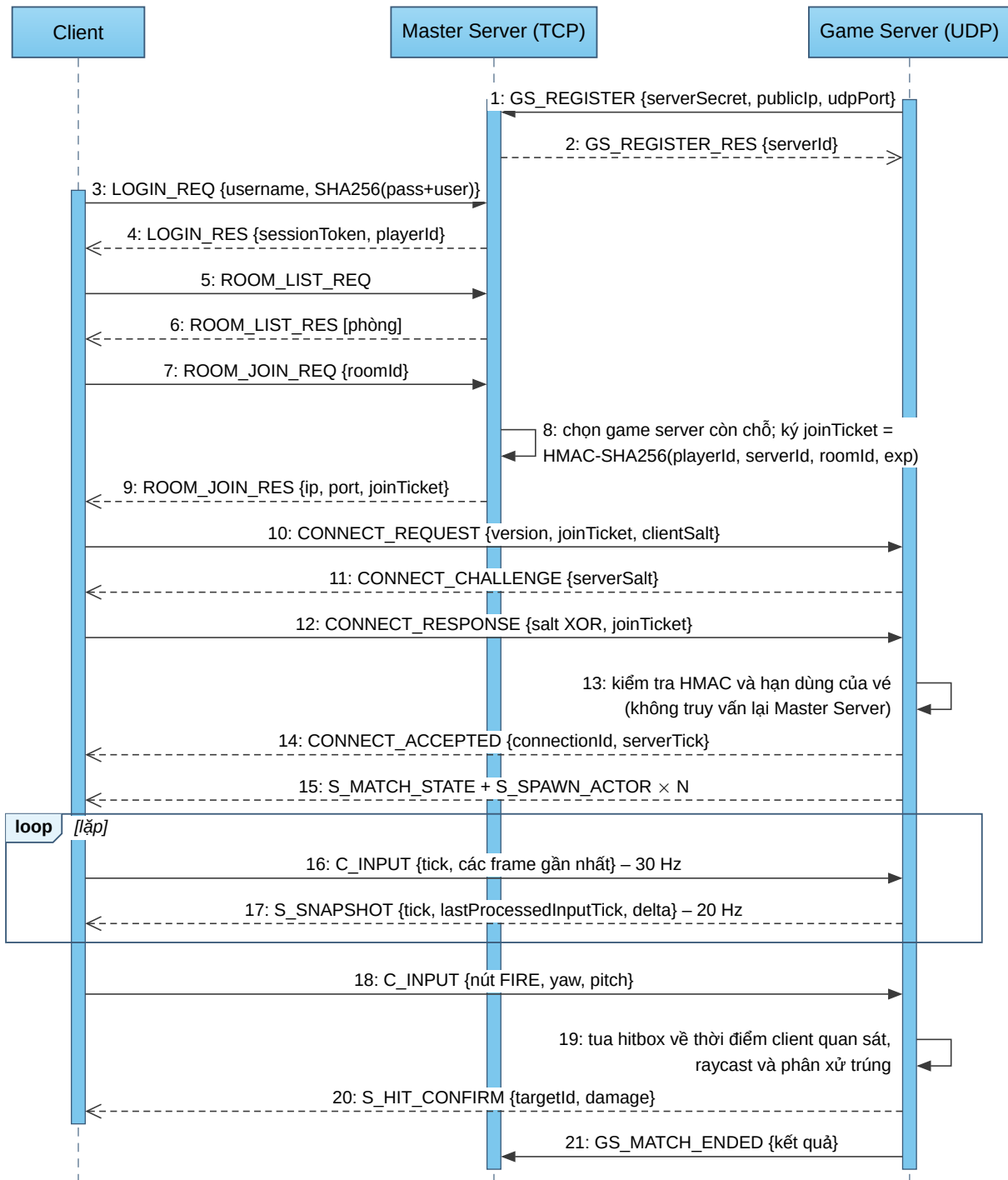
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

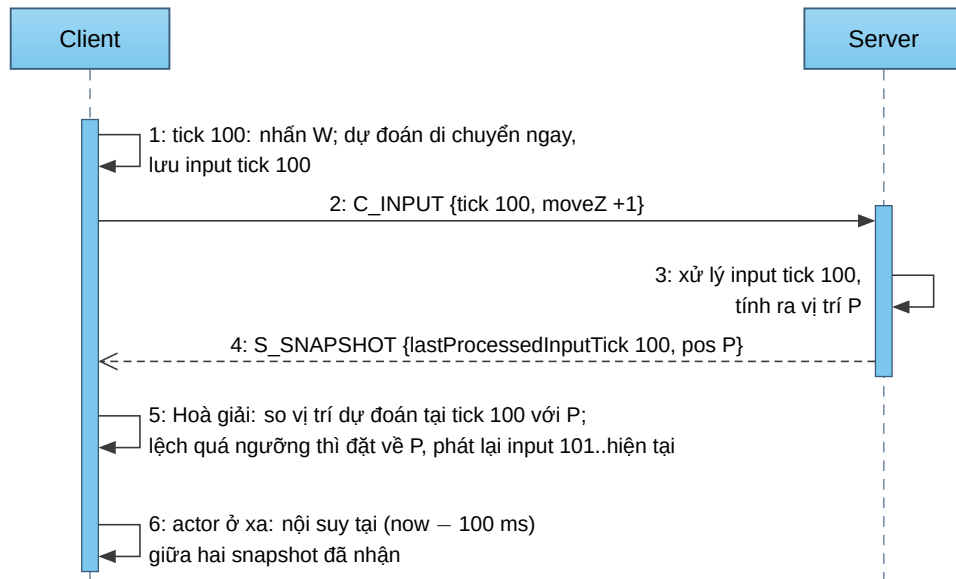
Vấn đề	Ai quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tản đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu – cuối



Hình 2.1. Luồng đầu – cuối: đăng nhập, vào phòng, kết nối game server, chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

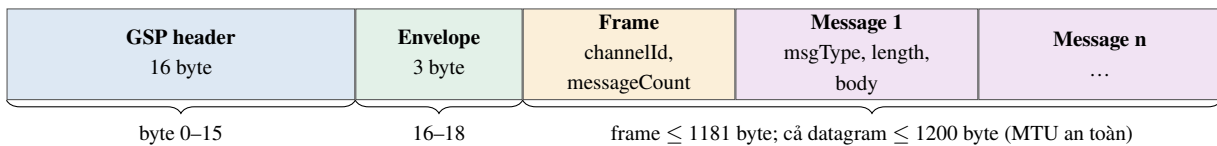


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng MovementSimulation ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP – cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tối khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nghẽn snapshot trên kênh 1 — ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP – khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket – cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng SHARED_SECRET; game server kiểm tra chữ ký bằng FixedTimeEquals và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo số lần FixedUpdate
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tĩnh	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thẩm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8–12 tuần
AD-3	Không theo đuổi tất định (lockstep)	Random rải rác 27 tệp; PhysX không tất định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** và `ProtocolConstants.cs` là nguồn sự thật duy nhất; `tools/SpecChecker` làm hỏng build nếu hai bên lệch nhau.
- **Tách người viết và người kiểm:** phân hệ Transport viết bit-packing, phân hệ Replication viết bộ test conformance bằng hex viết tay theo đặc tả.
- **CI** chạy trên Ubuntu và Windows: build coi cảnh báo là lỗi, chạy test, kiểm tra lệch đặc tả.

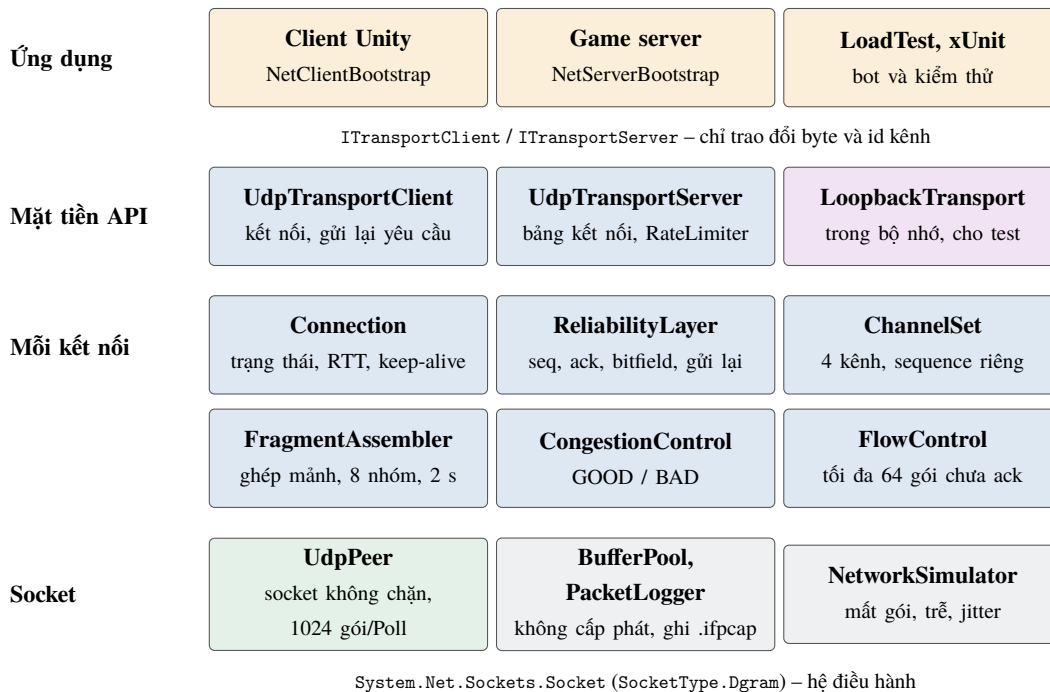
- **Definition of Done:** đã chạy và xem đầu ra, test phân hệ xanh, toàn bộ test xanh, Unity biên dịch sạch, đã merge và viết báo cáo phase.

II. NỘI DUNG CÁ NHÂN – Phân hệ tầng giao vận UDP (Transport).

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

Phân hệ Transport là tầng giao vận tin cậy được hiện thực trong không gian người dùng, trực tiếp trên socket UDP (System.Net.Sockets.Socket kiểu SocketType.Dgram), không sử dụng thư viện mạng bên thứ ba. Các chức năng mà UDP không cung cấp được bổ sung ở tầng này: đánh số thứ tự gói, xác nhận (ack) bằng bitfield, gửi lại, bốn kênh logic độc lập, phân mảnh và ghép mảnh, ước lượng RTT, điều khiển luồng, kiểm soát nghẽn, bắt tay chống giả mạo địa chỉ nguồn và bộ giả lập điều kiện mạng. Tầng này chỉ xử lý dữ liệu dưới dạng mảng byte, độc lập với ngữ nghĩa của tầng trên (snapshot, input hay chat), và không phụ thuộc UnityEngine nên dùng chung được cho client, server và công cụ kiểm thử.

3.1. Kiến trúc thư viện Transport



Hình 3.1. Biểu đồ thành phần thư viện Ironfront.Net.Transport

Thư viện được phân tầng theo trách nhiệm. UdpPeer là lớp duy nhất gọi API socket của hệ điều hành; lớp Connection và các thành phần bên dưới nó hiện thực các chức năng của một giao thức giao vận (tương ứng tầng 4 trong mô hình OSI) cho từng kết nối logic; UdpTransportClient/UdpTransportServer là mặt tiền API mà tầng ứng dụng sử dụng. Nhờ tách lớp socket, cùng một logic độ tin cậy chạy được trên socket thật, trên LoopbackTransport (trong bộ nhớ) và phía sau NetworkSimulator.

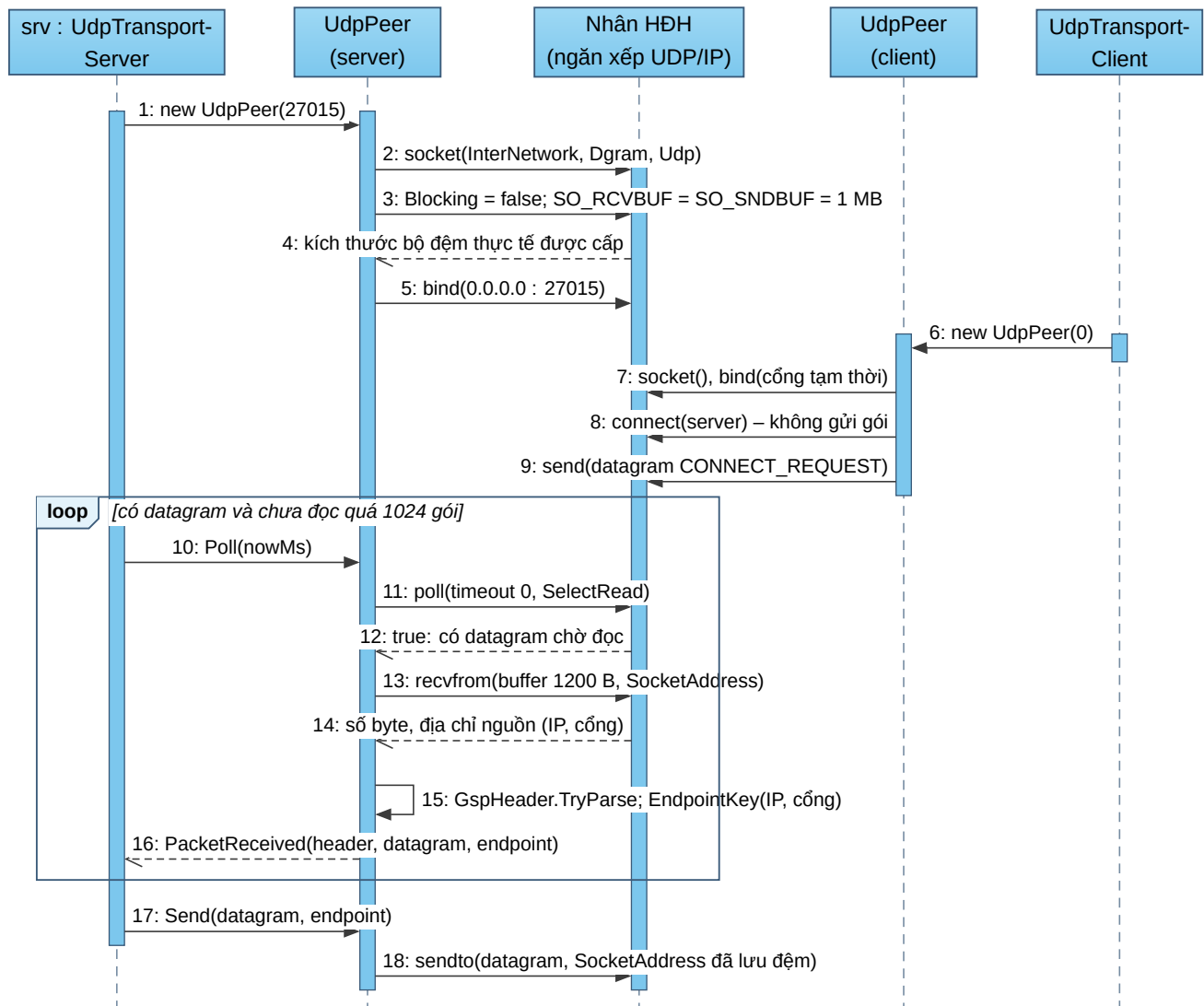
3.2. Lựa chọn UDP và mô hình lập trình socket

Lý do chọn UDP thay cho TCP. Dữ liệu của game bắn súng thời gian thực phần lớn là trạng thái có thời hạn sử dụng ngắn: một snapshot 50 ms trước đã vô giá trị khi snapshot mới tới. TCP đảm bảo giao đúng thứ tự trên một luồng byte duy nhất, nên khi một segment bị mất, mọi dữ liệu phía sau (kể cả snapshot mới hơn) bị giữ lại trong bộ đệm nhận của nhân cho tới

khi segment đó được gửi lại — hiện tượng **head-of-line blocking**. Với tỉ lệ mất gói vài phần trăm, độ trễ tăng thêm ít nhất một RTO cho toàn bộ luồng. UDP giao từng datagram độc lập, cho phép tầng ứng dụng tự quyết định dữ liệu nào cần tin cậy, dữ liệu nào chỉ cần bản mới nhất.

Đặc điểm	TCP (<code>SocketType.Stream</code>)	Transport trên UDP (<code>SocketType.Dgram</code>)
Đơn vị dữ liệu	Luồng byte, không có ranh giới bản tin	Datagram có ranh giới; mỗi <code>ReceiveFrom</code> trả đúng một gói
Thứ tự và tin cậy	Bắt buộc cho toàn bộ luồng	Chọn theo từng kênh (4 kênh, xem mục 4.4)
Mất một gói	Chặn mọi dữ liệu phía sau	Chỉ ảnh hưởng kênh tin cậy chứa gói đó
Trạng thái kết nối	Do nhân HĐH quản lý (bắt tay 3 bước, <code>accept</code>)	Do ứng dụng quản lý (<code>Connection</code> , bắt tay có cookie)
Số socket phía server	Một socket cho mỗi client	Một socket cho toàn bộ client, phân kênh theo (IP, cổng)
Gộp gói nhỏ	Thuật toán Nagle, cần tắt bằng <code>NoDelay</code>	Không có; mỗi <code>SendTo</code> là một datagram

Vòng đời socket. Hình 3.2 trình bày các lời gọi API socket thực tế của client và server. Server mở **một** socket UDP gắn vào cổng 27015 và nhận datagram của mọi client qua `ReceiveFrom`; địa chỉ nguồn trả về là căn cứ để xác định kết nối. Client dùng **socket UDP có kết nối** (`Connect` trên socket datagram): lời gọi này không gửi gói nào ra mạng, chỉ ghi địa chỉ đích mặc định vào socket, sau đó nhân hệ điều hành tự lọc bỏ datagram đến từ địa chỉ khác và client dùng được `Send/Receive` không cần tham số địa chỉ.



Hình 3.2. Biểu đồ tuần tự các lời gọi API socket UDP phía server và client

```

_socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp) {
    Blocking = false, ReceiveBufferSize = 1 << 20, SendBufferSize = 1 << 20 };
ReceiveBufferSize = _socket.ReceiveBufferSize; // read back what the OS granted
DisableIcmpPortUnreachable(); // Windows: SIO_UDP_CONNRESET
_socket.Bind(new IPEndPoint(IPAddress.Any, bindPort));
...
int drained = 0;
while (drained < MaxPacketsPerPoll && !_socket.Poll(0, SelectMode.SelectRead)) {
    drained++;
    byte[] buffer = _pool.Rent(); // 1200-byte pooled buffer
    received = _socket.ReceiveFrom(buffer.AsSpan(), SocketFlags.None, _receiveSocketAddress);
    _receiveEndpoint.Create(_receiveSocketAddress);
    if (GspHeader.TryParse(buffer.AsSpan(0, received), out GspHeader header))
        PacketReceived?.Invoke(header, new ReadOnlyMemory<byte>(buffer, 0, received), _receiveEndpoint);
    _pool.Return(buffer); // invalid header: dropped silently
}

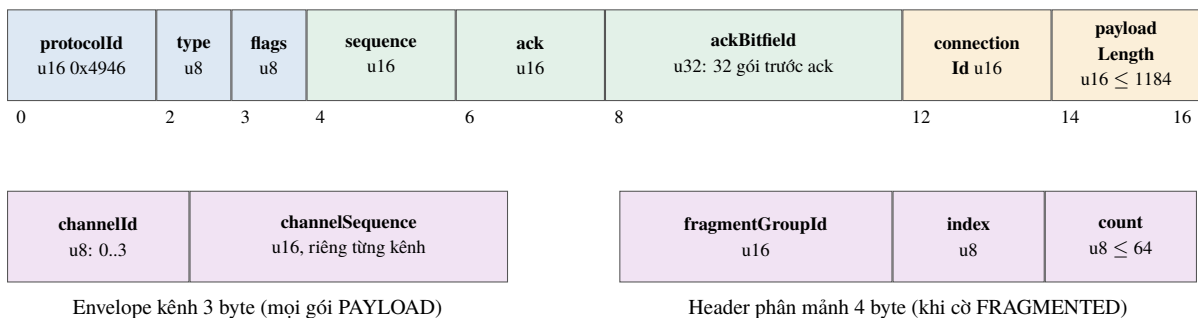
```

Đoạn mã rút gọn từ UdpPeer (hàm tạo và Poll)

Các quyết định lập trình socket trong UdpPeer:

- **Vào/ra không chặn, một luồng.** Socket đặt `Blocking=false`; mỗi tick, vòng lặp game gọi `Poll()`, lời gọi này kiểm tra trạng thái sẵn sàng bằng `Socket.Poll(0, SelectRead)` (tương đương `select()` với thời gian chờ 0) rồi đọc hết các datagram đang chờ. Không có luồng nhận riêng nên toàn bộ trạng thái kết nối chỉ được truy cập từ một luồng, không cần khoá và không có tranh chấp dữ liệu. Mẫu này phù hợp với vòng mô phỏng cố định 30 Hz của server và vòng Update của Unity.
- **Giới hạn số datagram mỗi lần đọc.** Mỗi `Poll` đọc tối đa 1024 datagram (khoảng 60 lần lưu lượng hợp lệ của 16 client ở 30 Hz). Nếu đọc tới khi bộ đệm rỗng, một đợt tấn công làm ngập gói có thể giữ luồng mô phỏng trong vòng nhận vô thời hạn và làm server ngừng mô phỏng; phần còn lại được đọc ở tick sau.
- **Bộ đệm socket.** Server yêu cầu `SO_RCVBUF` 1 MB (khoảng 870 datagram 1200 byte) để hấp thụ các đợt gói dồn dập. Linux giới hạn giá trị này bởi `net.core.rmem_max` (mặc định khoảng 208 KB) mà **không báo lỗi**; `UdpPeer` đọc lại kích thước thực tế và ghi cảnh báo, vì khi bộ đệm tràn, nhân hệ điều hành huỷ datagram trước khi mã ứng dụng kịp nhìn thấy.
- **ICMP Port Unreachable trên Windows.** Khi gửi tới một cổng đã đóng, Windows chuyển thông báo ICMP thành lỗi `WSAECONNRESET` ở lần `ReceiveFrom` kế tiếp, làm hỏng vòng nhận của server dùng chung một socket. `UdpPeer` tắt hành vi này bằng `IOControl(SIO_UDP_CONNRESET)`.
- **Phân kênh theo địa chỉ nguồn.** Server tra kết nối bằng khoá `EndpointKey` gồm địa chỉ IPv4 (u32) và cổng (u16) — kiểu giá trị, so sánh và băm không cấp phát bộ nhớ. Sau đó `connectionId` trong header phải khớp với kết nối của địa chỉ đó; sai lệch bị huỷ và đếm vào `PacketsWithBadConnectionId`.
- **Không cấp phát trong vòng gửi – nhận.** Trên .NET 8, `ReceiveFrom` ghi địa chỉ nguồn vào một `SocketAddress` tái sử dụng, và `SendTo` dùng `SocketAddress` đã tuần tự hoá sẵn cho từng đích, thay cho việc tạo `EndPoint` mới mỗi gói. Kết hợp với `BufferPool`, `Poll` và `Send` không cấp phát bộ nhớ sau giai đoạn khởi động, tránh các đợt dừng do bộ gom rác.

3.3. Định dạng gói tin GSP



Hình 3.3. Header GSP 16 byte (little-endian), envelope kênh và header phân mảnh

Trường	Giá trị
Bit flags	0 RELIABLE · 1 FRAGMENTED · 2 ORDERED · 3 COMPRESSED (dự phòng) · 4–7 phải bằng 0
packetType	0x01 CONNECT_REQUEST · 0x02 CONNECT_CHALLENGE · 0x03 CONNECT_RESPONSE · 0x04 CONNECT_ACCEPTED · 0x05 CONNECT_DENIED · 0x06 DISCONNECT · 0x07 KEEPALIVE · 0x10 PAYLOAD · 0x11 FRAGMENT

Kích thước datagram tối đa 1200 byte. Một datagram UDP vượt quá MTU của đường truyền bị tăng IP phân mảnh; mất một mảnh IP bất kỳ làm mất cả datagram, và nhiều tường lửa, thiết bị NAT huỷ luôn các mảnh IP. Giới hạn 1200 byte dữ liệu UDP cộng 28 byte header (IPv4 20 byte, UDP 8 byte) cho gói IP 1228 byte, nhỏ hơn MTU tối thiểu 1280 byte mà IPv6 bắt buộc mọi liên kết phải hỗ trợ và nhỏ hơn MTU 1492 byte của PPPoE hay MTU đã bị giảm của các đường hầm VPN. Sau 16 byte header GSP, phần dữ liệu còn tối đa 1184 byte; bản tin lớn hơn được phân mảnh ở tầng Transport (mục 4.5).

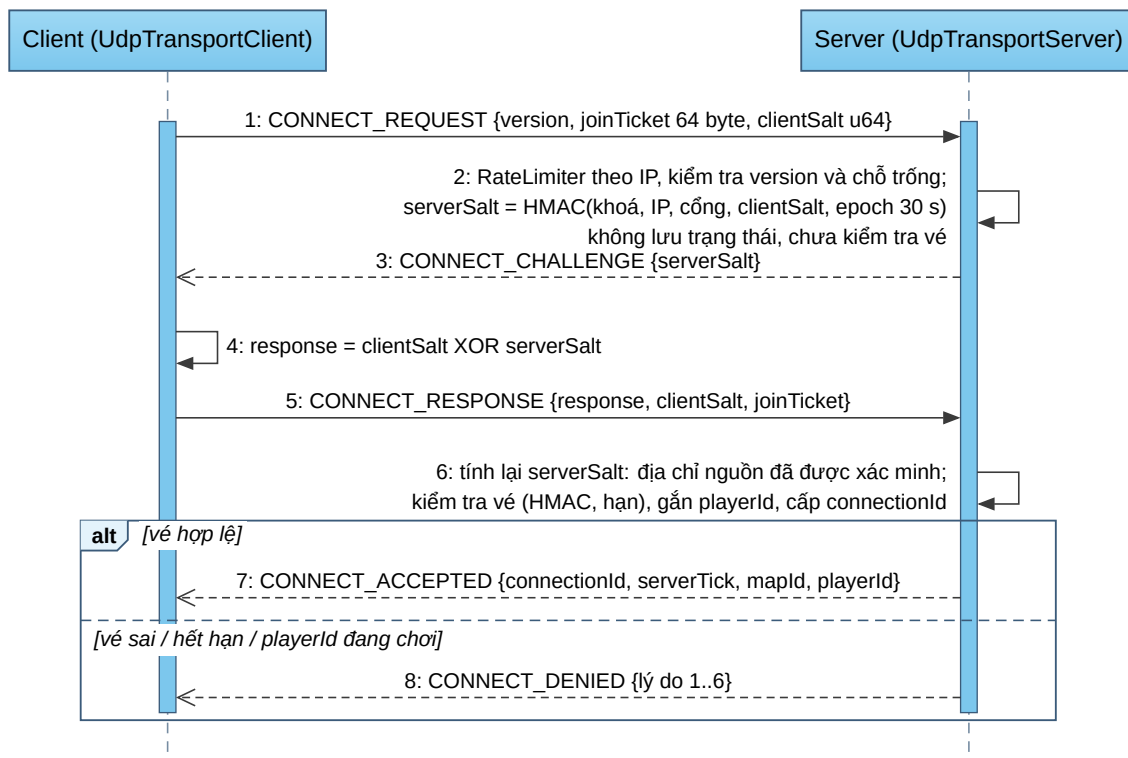
Tuần tự hoá nhị phân. Header có vị trí trường cố định, ghi theo thứ tự byte little-endian bằng các hàm `Endian.WriteU16LE/WriteU32LE` trên `Span<byte>`, không dùng `BinaryFormatter` hay `JSON`. Thứ tự byte được quy định tường minh trong đặc tả giao thức thay vì phụ thuộc kiến trúc máy, nên client và server trên các nền tảng khác nhau đọc cùng một chuỗi byte như nhau. Khi nhận, `TryParse` kiểm tra theo thứ tự: đủ 16 byte, `protocolId` bằng `0x4946` (loại bỏ gói lác từ ứng dụng khác), `payloadLength` ≤ 1184 và không vượt quá số byte thực nhận. Dữ liệu đến từ mạng được xem là không tin cậy: mọi độ dài đều được đối chiếu với kích thước bộ đệm trước khi đọc.

```
public bool TryWrite(Span<byte> dst) {
    if (dst.Length < Size) return false;
    Endian.WriteU16LE(dst, 0, ProtocolId); dst[2] = (byte)PacketType; dst[3] = (byte)Flags;
    Endian.WriteU16LE(dst, 4, Sequence); Endian.WriteU16LE(dst, 6, Ack);
    Endian.WriteU32LE(dst, 8, AckBitfield); Endian.WriteU16LE(dst, 12, ConnectionId);
    Endian.WriteU16LE(dst, 14, PayloadLength);
    return true;
}

public static bool TryParse(ReadOnlySpan<byte> src, out GspHeader header) {
    header = default;
    if (src.Length < Size) return false;
    if (Endian.ReadU16LE(src, 0) != ProtocolConstants.PROTOCOL_ID) return false;
    ushort payloadLength = Endian.ReadU16LE(src, 14);
    if (payloadLength > ProtocolConstants.MAX_PAYLOAD) return false;
    if (src.Length < Size + payloadLength) return false;
    ...
}
```

Đoạn mã rút gọn từ `GspHeader`: ghi và phân tích header 16 byte

3.4. Bắt tay không lưu trạng thái



Hình 3.4. Bắt tay challenge – response dùng cookie HMAC: server không cấp phát tài nguyên cho địa chỉ chưa được xác minh

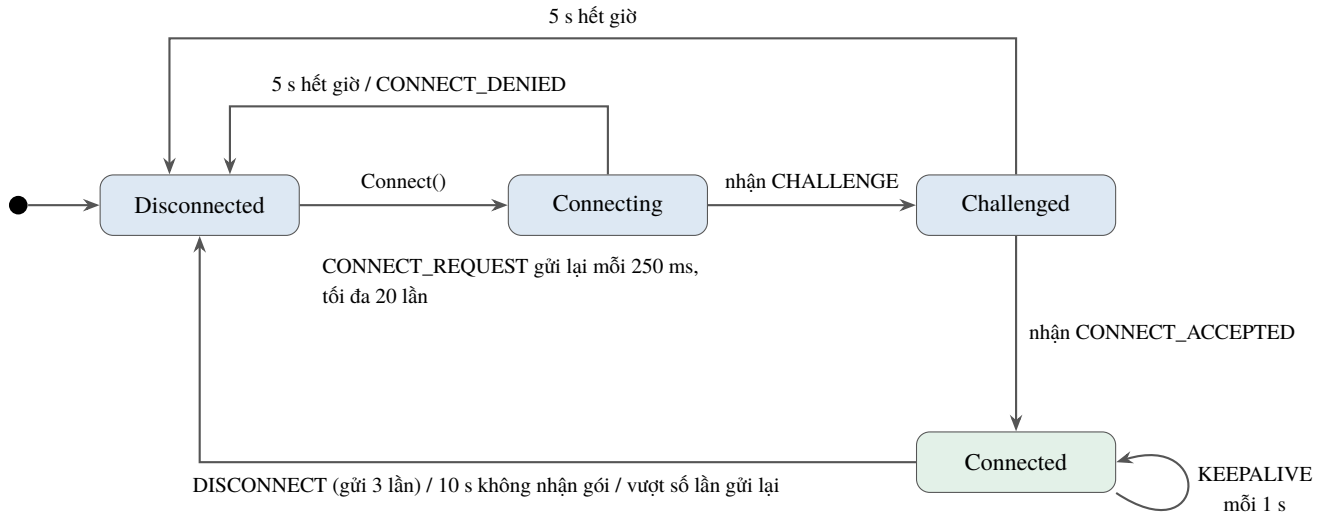
Với UDP, địa chỉ nguồn trong gói tin có thể bị giả mạo tùy ý vì không có bắt tay ở tầng giao vận. Cơ chế trên áp dụng nguyên lý của **SYN cookie** trong TCP: thay vì lưu một bản ghi chờ cho mỗi yêu cầu, server mã hoá thông tin cần kiểm tra

vào chính giá trị gửi đi và tính lại khi nhận phản hồi:

$$serverSalt = \text{HMAC-SHA256}(K, IP \parallel port \parallel clientSalt \parallel \lfloor t/30 \text{ s} \rfloor)[0..7], \quad response = clientSalt \oplus serverSalt$$

Khoá K 32 byte sinh ngẫu nhiên khi server khởi động và không rời khỏi tiến trình. Chỉ bên thực sự nhận được gói challenge tại địa chỉ đã khai mới tính được $response$ đúng, nên một `CONNECT_RESPONSE` hợp lệ chứng minh quyền sở hữu địa chỉ nguồn. Thành phần epoch giới hạn thời gian dùng lại cookie bị bắt được; server chấp nhận epoch hiện tại và epoch liền trước để bắt tay diễn ra đúng lúc chuyển epoch không bị từ chối. Giá trị $serverSalt$ bằng 0 được thay bằng 1, vì khi đó $response$ trùng với $clientSalt$ và bất kỳ bên quan sát nào cũng trả lời được.

3.5. Máy trạng thái kết nối



Hình 3.5. Máy trạng thái ConnectionState của một kết nối

3.6. Cơ chế ack bằng bitfield và gửi lại

seq 103 đã nhận	seq 102 MẤT	seq 101 đã nhận	seq 100 MẤT	seq 99 đã nhận	seq 98 đã nhận	seq 97 đã nhận
ack = 103	bit0 = 0	bit1 = 1	bit2 = 0	bit3 = 1	bit4 = 1	bit5 = 1

Hình 3.6. Ví dụ ack bitfield: $ack = 103$, $ackBitfield = \dots 111010$; bên gửi xác định được gói 102 và 100 bị mất. Mỗi header mang trạng thái nhận của 33 gói nên không cần gói ACK riêng

Số học sequence modulo 2^{16} . Trường sequence 16 bit quay vòng về 0 sau 65 535, tức khoảng 36 phút ở 30 gói/giây. Phép so sánh thông thường $a > b$ sai tại điểm quay vòng (gói 0 mới hơn gói 65 535). Mọi phép so sánh sequence trong mã nguồn đi qua lớp `SequenceMath`, dùng khoảng cách có dấu theo số học modulo:

$$d(a, b) = ((a - b) \bmod 2^{16}) \text{ diễn giải là số nguyên có dấu 16 bit, } a \text{ mới hơn } b \iff 0 < d(a, b) \leq 2^{15}$$

Trong C#, $d(a, b)$ được tính bằng một phép ép kiểu (short) $(a-b)$: phép trừ tràn tự nhiên theo modulo 2^{16} và ép sang short cho kết quả trong khoảng $[-32768, 32767]$. Hai sequence cách nhau dưới nửa không gian số được so sánh đúng bất kể quay vòng; bộ kiểm thử có trường hợp biên $65\,535 \rightarrow 0$.

Cập nhật cửa sổ nhận. Bên nhận giữ `remoteSequence` (sequence mới nhất đã nhận) và thanh ghi 32 bit `receivedBitfield`, trong đó bit i bằng 1 nếu đã nhận gói `remoteSequence - 1 - i`. Khi gói mới hơn tới với khoảng cách d , thanh ghi được dịch trái d bit và bật bit của `remoteSequence` cũ; khi gói cũ hơn tới muộn (đảo thứ tự) trong phạm vi 32, chỉ bật bit tương ứng. Bên gửi duyệt ack và 32 bit để xác nhận từng gói trong vòng đệm gửi 1024 ô (chỉ số = `sequence mod 1024`).

```
public void OnPacketReceived(ushort sequence) {
    int distance = SequenceMath.Distance(sequence, _remoteSequence); // (short)(a - b)
    if (distance > 0) { // newer packet
        uint shifted = distance >= 32 ? 0u : _receivedBitfield << distance;
        uint newestHistory = distance > 32 ? 1u << (distance - 1);
        _receivedBitfield = shifted | newestHistory;
        _remoteSequence = sequence;
        return;
    }
    int behind = -distance; // late, reordered packet
    if (behind >= 1 && behind <= 32) _receivedBitfield |= 1u << (behind - 1);
}

public void ProcessIncomingAck(ushort ack, uint bitfield, double nowMs) {
    AckPacket(ack, nowMs);
    for (int bit = 0; bit < 32; bit++)
        if ((bitfield & (1u << bit)) != 0) AckPacket((ushort)(ack - 1 - bit), nowMs);
}
```

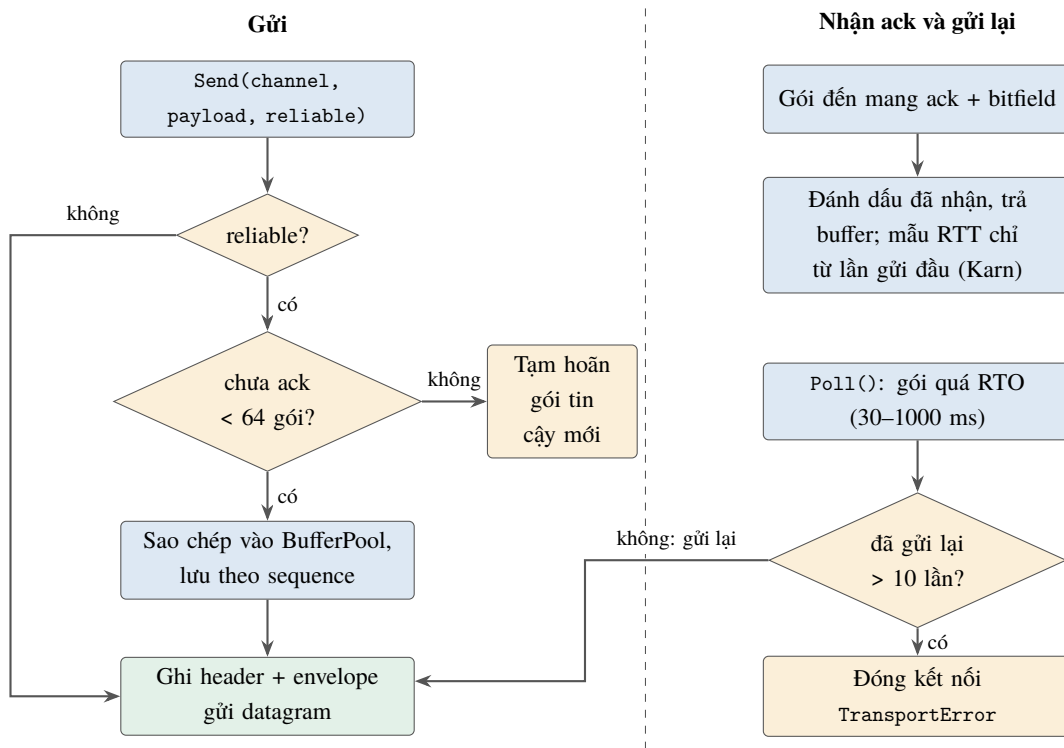
Đoạn mã rút gọn từ ReliabilityLayer: cập nhật cửa sổ nhận và xử lý ack

Ước lượng RTT và thời gian chờ gửi lại (RTO). Mỗi lần một gói được ack, thời gian từ lúc gửi tới lúc nhận ack là một mẫu RTT R . Theo **thuật toán Karn**, gói đã từng được gửi lại không cho mẫu RTT vì không xác định được ack ứng với lần gửi nào. RTT làm mượt (SRTT) và độ dao động (jitter) được cập nhật theo trung bình trượt hàm mũ (EWMA), cùng dạng với thuật toán Jacobson/Karels trong TCP:

$$jitter \leftarrow jitter + 0,25 (|R - SRTT| - jitter), \quad SRTT \leftarrow 0,9 SRTT + 0,1 R$$

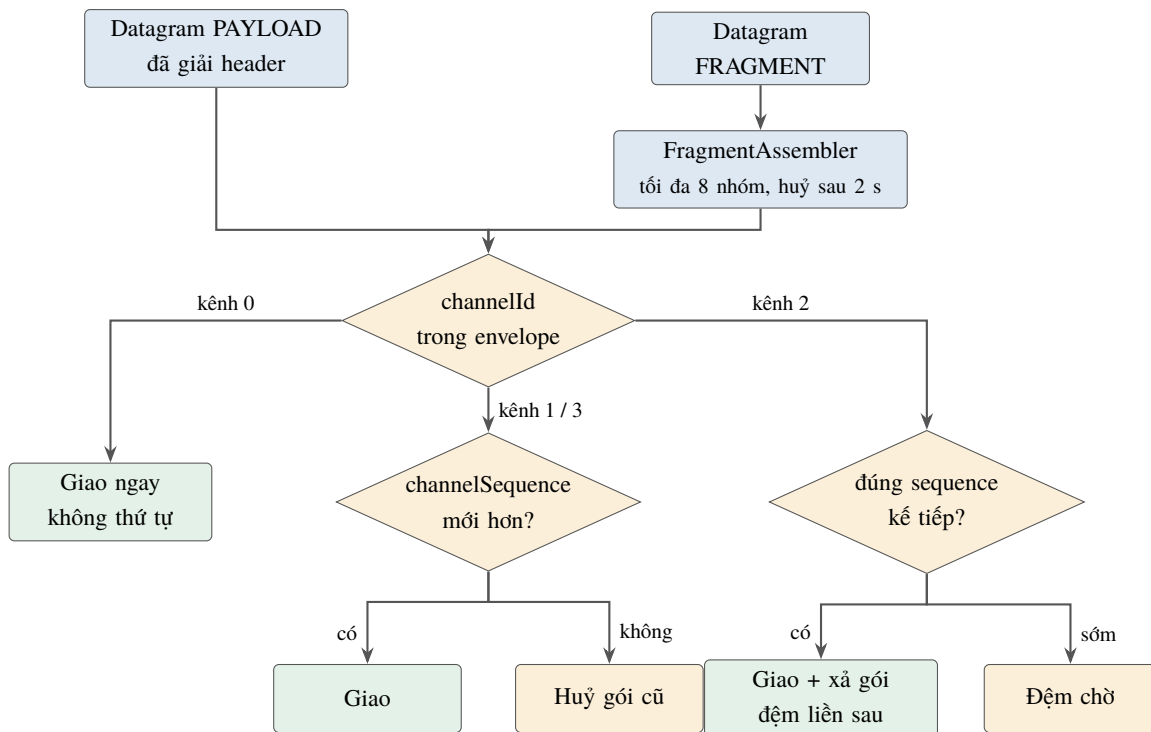
$$RTO = \min(1000, \max(30, 1,5 SRTT + 4 jitter)) \text{ ms}$$

Hệ số nhân 1,5 (thấp hơn mức của TCP) phù hợp với dữ liệu game cần gửi lại nhanh; cận dưới 30 ms tránh gửi lại dồn dập trên mạng LAN, cận trên 1000 ms giới hạn thời gian treo của kênh tin cậy.

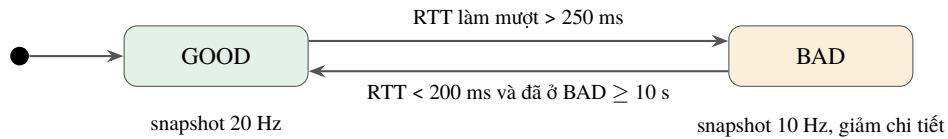


Hình 3.7. Luồng gửi tin cậy, xử lý ack và gửi lại trong ReliabilityLayer

3.7. Kênh, phân mảnh và kiểm soát nghẽn

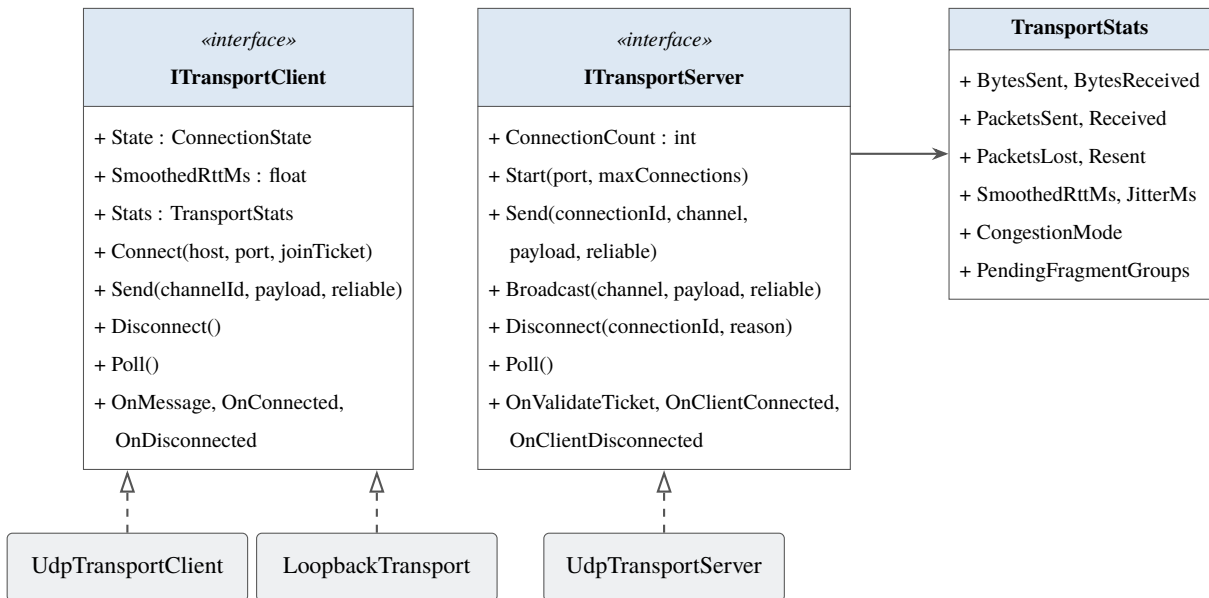


Hình 3.8. Xử lý phía nhận theo kênh: kênh 2 chờ đúng thứ tự, kênh 1 và 3 bỏ gói cũ



Hình 3.9. Kiểm soát nghẽn hai chế độ có trễ (hysteresis) 250/200 ms; RTT làm mượt theo EWMA 0,9/0,1

3.8. Giao diện lập trình và giao diện chẩn đoán



Hình 3.10. Giao diện lập trình công khai của Transport (giữ ổn định từ tuần đầu của dự án)

Giao diện chỉ nhận và trả `ReadOnlySpan<byte>/ReadOnlyMemory<byte>` cùng id kênh, không chứa kiểu dữ liệu nào của tầng game. Mọi sự kiện (`OnMessage`, `OnConnected`...) được phát đồng bộ bên trong `Poll()` trên luồng gọi, nên tầng ứng dụng không phải xử lý đồng bộ hoá giữa các luồng.

<pre> Transport Connected RTT 48.2 ms jitter 3.1 ms loss up 0.4% down 1.2% rate up 0.9 KB/s down 1.7 KB/s congestion GOOD fragments 0 pool 12 packets sent 18422 recv 12291 lost 147 resent 52 </pre>	<pre> \$ dotnet run --project Ironfront.Tools.PacketReplay \ -- capture.ifpcap --conn 3 --from 12.0 --to 15.0 t=12.004 OUT PAYLOAD seq=4412 ack=3380 ch=1 len=87 t=12.021 IN PAYLOAD seq=3381 ack=4411 ch=3 len=41 t=12.062 IN KEEPALIVE seq=3382 ack=4413 rtt~41ms gap: seq 4414 not acked after 3 datagrams summary: 118 out, 94 in, 2 gaps, rtt p50 44ms </pre>
---	--

Hình 3.11. Giao diện chẩn đoán: văn bản overlay do `TransportDiagnosticsFormatter` sinh (trái) và minh hoạ đầu ra công cụ phát lại gói (phải); các số liệu chỉ minh hoạ bộ cục

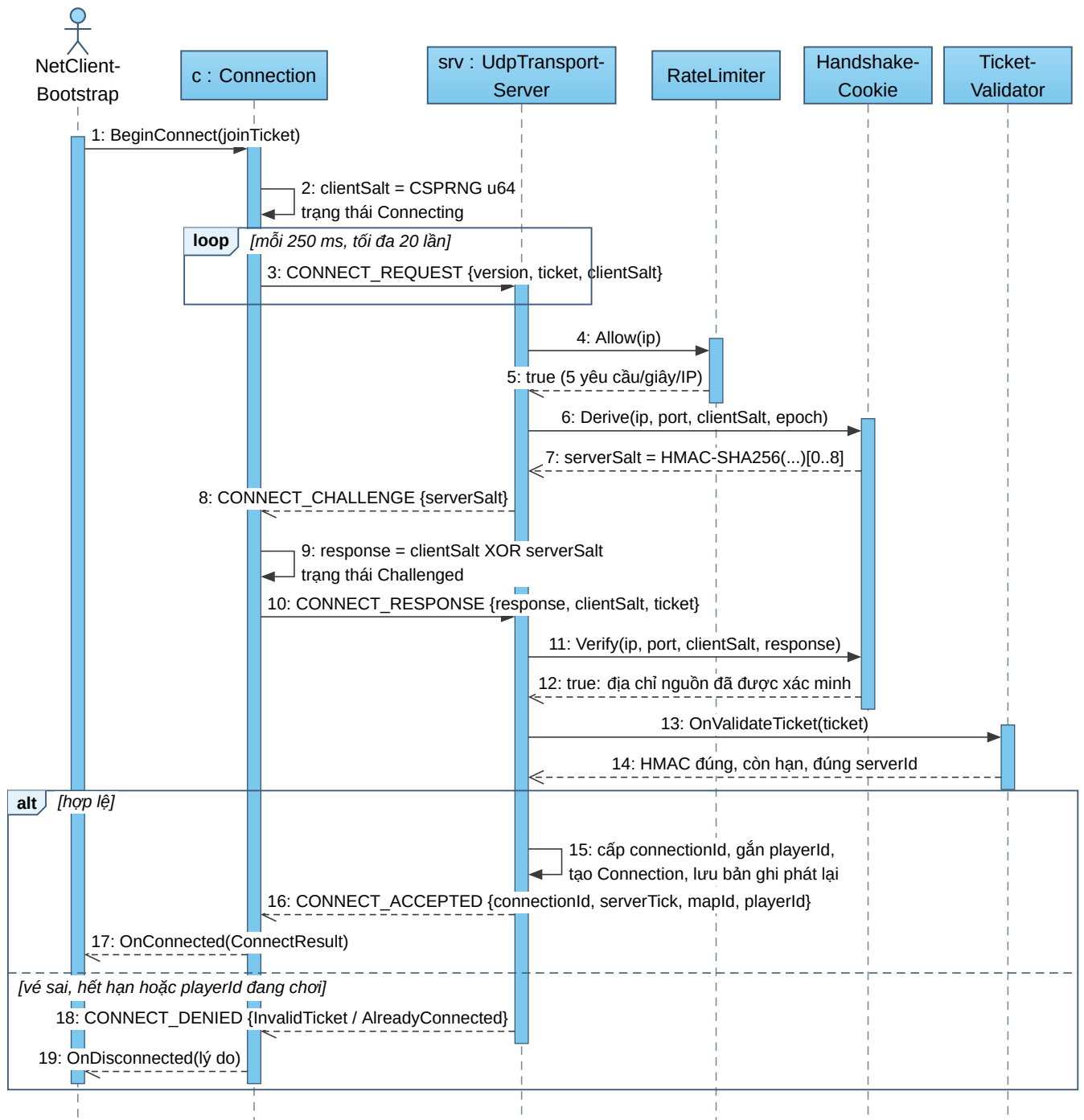
4. Luồng xử lý và giải pháp đã ứng dụng

Mục này trình bày lần lượt các đầu việc của phân hệ theo bản mô tả đề tài: thiết lập kết nối UDP, header 16 byte, bắt tay có cookie, ack bằng bitfield và gửi lại, bốn kênh truyền, phân mảnh và ghép mảnh, kiểm soát nghẽn, duy trì kết nối, bộ giả lập mạng và công cụ đo. Với mỗi đầu việc, báo cáo trình bày **luồng xử lý** (biểu đồ tuần tự sử dụng đúng tên lớp trong mã

nguồn), **vấn đề kỹ thuật** và **giải pháp đã áp dụng**. Các tham số lấy trực tiếp từ mã nguồn `Ironfront.Net.Transport` và `ProtocolConstants`.

4.1. Luồng thiết lập kết nối UDP có xác thực

Bài toán. UDP không có khái niệm kết nối: bất kỳ máy nào cũng gửi được datagram tới cổng 27015 với địa chỉ nguồn giả mạo. Nếu server cấp phát bộ nhớ, sinh số ngẫu nhiên hoặc kiểm tra chữ ký ngay khi nhận `CONNECT_REQUEST`, kẻ tấn công có thể gửi hàng loạt yêu cầu với địa chỉ nguồn giả để làm đầy bảng kết nối đang chờ và khiến các client hợp lệ bị loại khỏi quá trình bắt tay. Đồng thời, server phải xác thực được client kết nối đúng là người đã được Master Server cấp vé vào trận.



Hình 4.1. Biểu đồ tuần tự thiết lập kết nối: server không lưu trạng thái cho tới khi địa chỉ nguồn được xác minh

Diễn giải luồng.

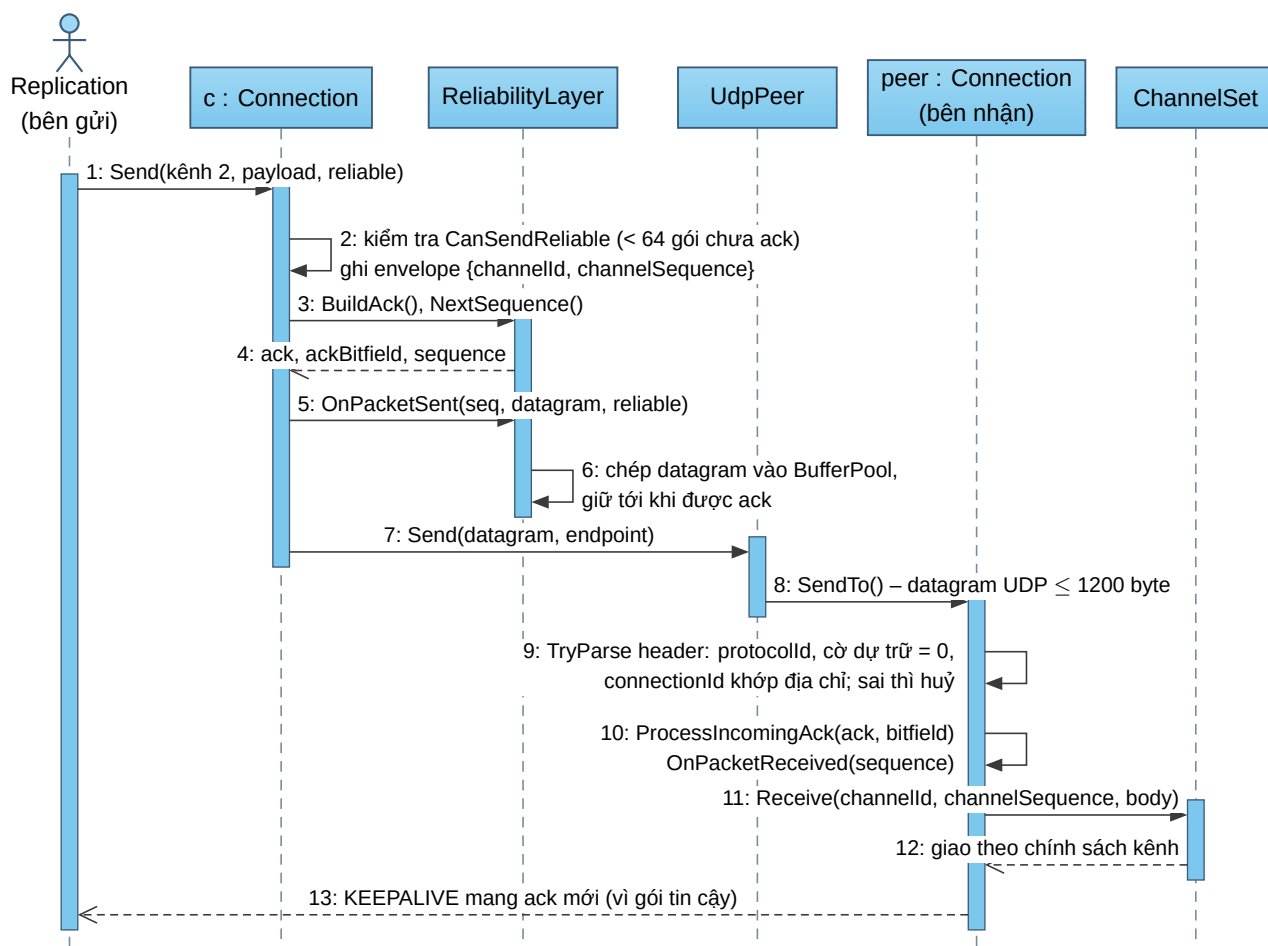
- Client sinh `clientSalt` bằng bộ sinh số ngẫu nhiên an toàn mật mã (`RandomNumberGenerator`) và gửi `CONNECT_REQUEST`. Gói có kích thước cố định; gói sai kích thước bị hủy mà không phản hồi. Vì chính gói yêu cầu cũng có thể bị mất trên đường truyền, client gửi lại mỗi 250 ms, tối đa 20 lần (5 giây).
- Server kiểm tra theo thứ tự chi phí tăng dần: giới hạn tần suất theo IP, phiên bản giao thức, địa chỉ đã có kết nối hay chưa, còn chỗ trống hay không. Server **không lưu trạng thái** cho yêu cầu này mà tính `serverSalt` bằng HMAC-SHA256 trên (địa chỉ, cổng, `clientSalt`, epoch 30 s) và gửi `CONNECT_CHALLENGE` về địa chỉ nguồn đã khai.

- Chỉ client thực sự sở hữu địa chỉ đó mới nhận được challenge và tính được `clientSalt XOR serverSalt`. Khi `CONNECT_RESPONSE` tới, server tính lại cookie (chấp nhận epoch hiện tại và epoch liền trước) để xác minh; từ thời điểm này địa chỉ nguồn được xem là đáng tin cậy.
- Chỉ sau bước xác minh, server mới thực hiện các thao tác tốn tài nguyên: kiểm tra HMAC của `joinTicket`, đọc `playerId` và từ chối nếu `playerId` đó đang có kết nối khác. Nếu hợp lệ, server lấy một `connectionId` từ hàng đợi id còn trống, tạo `Connection` và gửi `CONNECT_ACCEPTED`.
- `CONNECT_ACCEPTED` cũng có thể bị mất. Server giữ một bản ghi nhỏ theo địa chỉ; nếu client gửi lại `CONNECT_RESPONSE` với đúng đáp án, server gửi lại `CONNECT_ACCEPTED` đã cập thay vì tạo kết nối thứ hai (thao tác bất tay có tính lũy đẳng).

Vấn đề	Giải pháp đã áp dụng
Tấn công làm ngập <code>CONNECT_REQUEST</code> với địa chỉ nguồn giả mạo, làm đầy bảng kết nối đang chờ	Cookie không trạng thái theo nguyên lý SYN cookie (HandshakeCookie): <code>serverSalt</code> là HMAC của địa chỉ, cổng, <code>clientSalt</code> và epoch 30 s, được tính lại khi cần và không bao giờ lưu
Server bị lợi dụng để khuếch đại lưu lượng (reflection/amplification)	Challenge là một gói nhỏ gửi về địa chỉ đã khai; <code>RateLimiter</code> giới hạn 5 yêu cầu/giây/IP, bảng theo dõi tối đa 10 000 IP và tự dọn sau 10 s
Một vé bị lộ được dùng cho nhiều kết nối	Ràng buộc <code>playerId</code> trong vé với <code>connectionId</code> ; vé thứ hai bị từ chối với lý do <code>AlreadyConnected</code> , phiên đang chơi không bị ngắt
Gói <code>CONNECT_ACCEPTED</code> bị mất	Bản ghi phát lại theo địa chỉ (<code>AcceptedHandshake</code>), hết hạn sau 10 s
Con trỏ ack bị khởi tạo sai từ bộ đếm sequence của gói điều khiển	Gói bắt tay không cập nhật con trỏ ack; gói dữ liệu đầu tiên mới khởi tạo nó

4.2. Luồng gửi và nhận một bản tin

Tầng trên (Replication) chỉ gọi `Send(channelId, payload, reliable)` và nhận lại `OnMessage(payload)`. Toàn bộ các bước ở giữa — đánh số, dựng header, lưu bản sao để gửi lại, ghi vào socket, phân loại theo kênh — do Transport đảm nhiệm. Toàn bộ xử lý chạy trên luồng gọi `Poll()`, không tạo luồng riêng, nên không cần khoá.

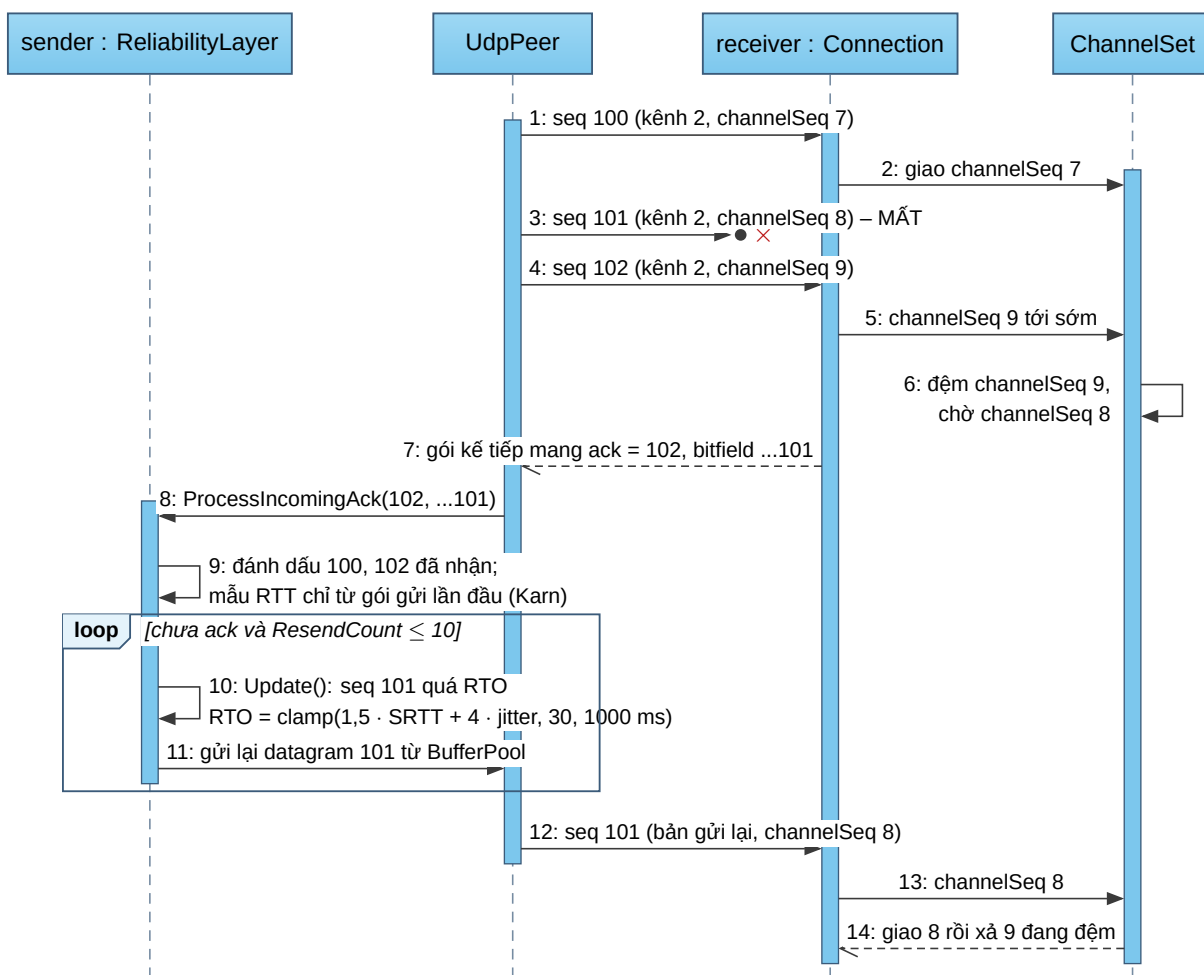


Hình 4.2. Biểu đồ tuần tự gửi – nhận một bản tin tin cậy qua kênh 2

Các giải pháp tối ưu đường xử lý gói tin.

- **Ack đính kèm (piggyback).** Mọi header đều mang ack và ackBitfield 32 bit, nên một gói xác nhận được 33 gói cùng lúc và không cần gói ACK riêng. Riêng khi nhận gói tin cậy, bên nhận gửi ngay một KEEPALIVE để bên gửi không phải chờ tới chu kỳ keep-alive 1 giây mới nhận được ack.
- **Không cấp phát bộ nhớ trong vòng gửi – nhận.** Mọi mảng byte lấy từ BufferPool với kích thước cố định 1200 byte. Đo trên .NET 8 Release: BufferPool 10,4 ns/lần, ArrayPool.Shared 17,4 ns, new byte[] 16,7 ns; sau giai đoạn khởi động, Poll và Send cấp phát 0 byte, loại bỏ các lần dừng do bộ gom rác trong vòng mô phỏng.
- **Giới hạn số gói mỗi lần nhận.** Mỗi lần Poll đọc tối đa 1024 gói; phần còn lại được xử lý ở lượt sau (33 ms). Nếu không giới hạn, khi bị tấn công làm ngập gói, luồng mô phỏng bị giữ trong vòng nhận và server ngừng mô phỏng.
- **Bộ đệm socket 1 MB và cảnh báo khi hệ điều hành giới hạn.** Linux mặc định giới hạn rmem_max khoảng 208 KB mà không báo lỗi; UdpPeer đọc lại kích thước thực tế và ghi cảnh báo. Trên Windows, tắt SIO_UDP_CONNRESET để thông báo ICMP “port unreachable” không làm lỗi lần nhận kế tiếp.
- **Không phản hồi gói không hợp lệ.** Header sai, connectionId không khớp địa chỉ hoặc cờ dự trữ khác 0 đều bị hủy mà không phản hồi; phản hồi các gói này sẽ biến server thành công cụ dò quét cổng.

4.3. Luồng phát hiện mất gói và gửi lại



Hình 4.3. Biểu đồ tuần tự mất gói trên kênh tin cậy: bitfield chỉ ra gói mất, RTO kích hoạt gửi lại, kênh giữ đúng thứ tự

Vấn đề	Giải pháp đã áp dụng
Xác định gói bị mất mà không tốn gói ACK riêng	Cửa sổ nhận 32 bit dịch theo khoảng cách sequence; bên gửi duyệt ack và 32 bit để xác nhận từng gói (ProcessIncomingAck)
Số thứ tự 16 bit quay vòng sau khoảng 36 phút ở 30 gói/giây	So sánh bằng SequenceMath.IsNewer / Distance theo số học modulo 2^{16} , không dùng phép so sánh thông thường; có kiểm thử biên tại $65535 \rightarrow 0$
RTT bị đo sai khi gói đã được gửi lại	Thuật toán Karn: chỉ lấy mẫu RTT khi gói được ack mà chưa từng gửi lại; SRTT làm mượt EWMA 0,9/0,1, jitter theo hệ số 0,25
Thời gian chờ gửi lại quá dài hoặc gửi lại dồn dập	$RTO_{động} = 1,5 \times SRTT + 4 \times jitter$, giới hạn trong [30; 1000] ms
Gói tin cậy mất vĩnh viễn làm kênh có thứ tự bị treo	Quá 10 lần gửi lại hoặc trùng ô vòng đệm 1024 thì đóng kết nối với TransportError, thay vì duy trì một kết nối không còn giao được dữ liệu
Bên nhận quá tải	Điều khiển luồng: tối đa 64 gói tin cậy chưa ack; KEEPALIVE thông báo số gói đang chờ và mức áp lực, bên gửi tạm dừng gói tin cậy mới khi áp lực > 80%

4.4. Bốn kênh truyền và luồng phân loại khi nhận

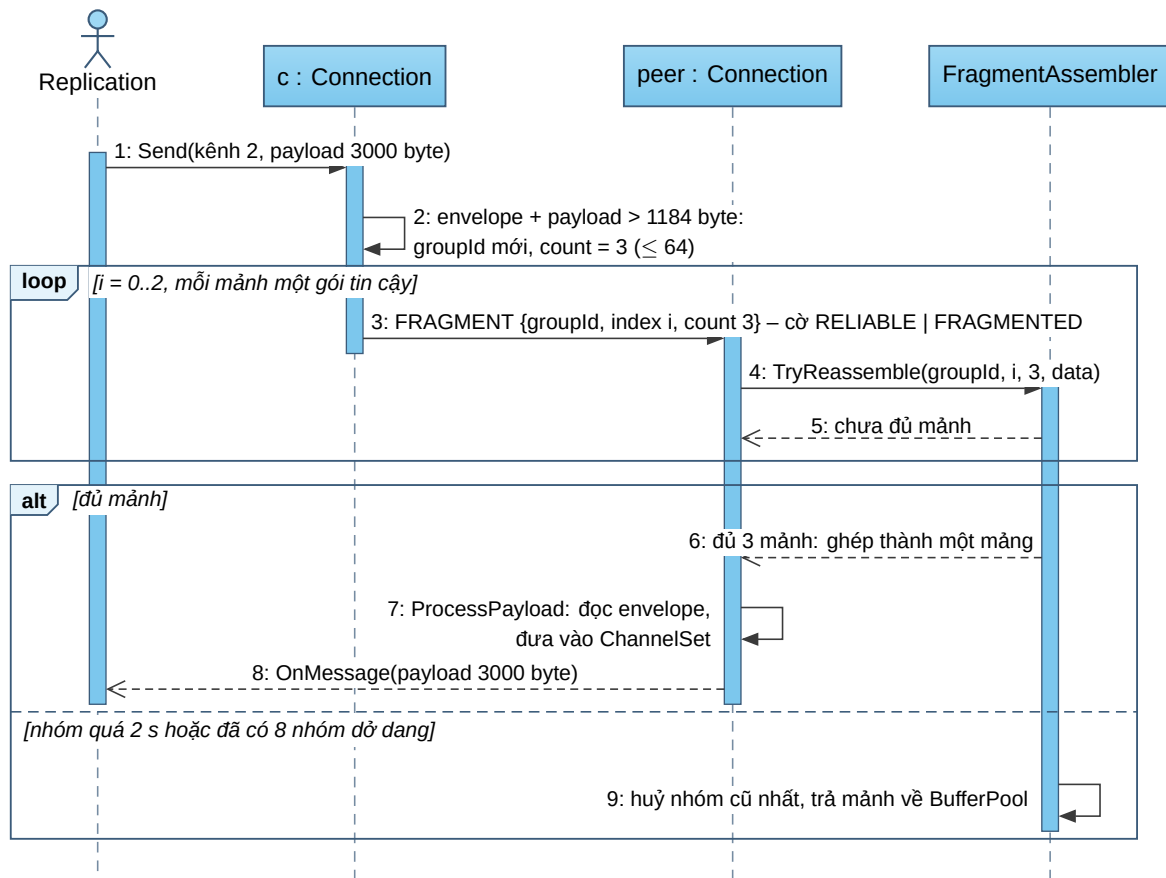
Mỗi gói PAYLOAD có envelope 3 byte gồm channelId và channelSequence **riêng từng kênh**, tách biệt với sequence của kết nối. Nhờ vậy, các gói KEEPALIVE hay gói của kênh khác không làm thay đổi thứ tự đánh giá snapshot trên kênh 1.

Kênh	Đảm bảo	Cách xử lý phía nhận (ChannelSet.Receive)	Dữ liệu
0	Không tin cậy, không thứ tự	Giao ngay	Ping/pong
1	Không tin cậy, có thứ tự	Giao nếu channelSequence mới hơn gói đã giao; gói cũ bị huỷ và đếm StalePacketsDropped	Snapshot server → client
2	Tin cậy, có thứ tự	Cửa sổ 256 ô: gói đúng thứ tự thì giao và xả các gói liền sau đang đệm; gói đến sớm được chép vào bộ đệm; gói trùng hoặc ngoài cửa sổ bị bỏ	Sự kiện, spawn, chat
3	Không tin cậy, có thứ tự	Như kênh 1 nhưng bộ đếm riêng	Input client → server

Giải pháp chống head-of-line blocking: độ tin cậy được áp dụng **theo kênh**. Thí nghiệm --phase4-report cho thấy khi kênh 2 đang thiếu sequence 0, snapshot kênh 1 vẫn được giao ngay (1 gói), còn kênh 2 đệm gói sau và chỉ giao cả hai khi gói thiếu tới. Nếu dùng TCP, snapshot phải chờ gói sự kiện bị mất được gửi lại.

4.5. Luồng cắt và ghép mảnh bản tin lớn

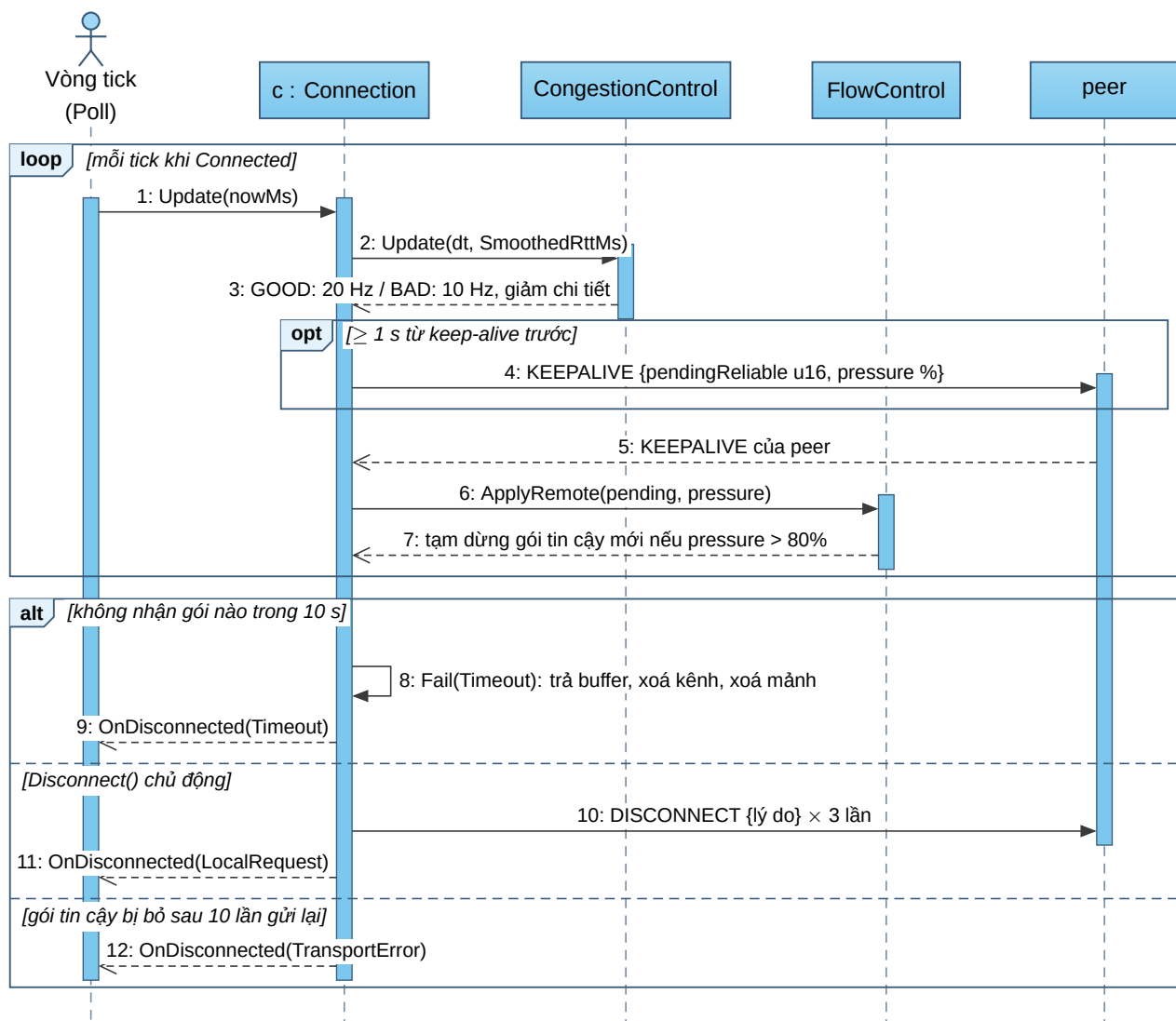
Datagram được giới hạn 1200 byte để tránh phân mảnh ở tầng IP trên các đường truyền có PPPoE, VPN hoặc đường hầm (payload tối đa 1184 byte sau header 16 byte, xem mục 3.3). Bản tin lớn hơn — ví dụ danh sách người chơi khi mới vào trận — được phân mảnh ở tầng Transport, nơi từng mảnh được đánh số và gửi lại riêng khi bị mất.



Hình 4.4. Biểu đồ tuần tự cắt và ghép mảnh với bộ nhớ có giới hạn

Giải pháp. (1) Mảnh luôn được gửi **tin cậy** kể cả khi tầng trên yêu cầu không tin cậy, vì thiếu một mảnh thì toàn bộ bản tin không sử dụng được. (2) Giới hạn bộ nhớ cho dữ liệu do bên gửi kiểm soát: tối đa 64 mảnh mỗi nhóm, 8 nhóm dở dang mỗi kết nối, nhóm quá 2 giây bị huỷ, và nếu count của một mảnh khác với nhóm thì huỷ cả nhóm. (3) Trước khi cắt, kiểm tra tổng số gói tin cậy chưa ack và số mảnh không vượt quá 64, tránh một bản tin lớn chiếm toàn bộ cửa sổ gửi. Kiểm thử: bản tin 20 KB được ghép đúng từng byte khi các mảnh đến ngược thứ tự.

4.6. Luồng duy trì kết nối, kiểm soát nghẽn và ngắt kết nối



Hình 4.5. Biểu đồ tuần tự duy trì kết nối, điều khiển luồng và các trường hợp kết thúc kết nối

Vấn đề	Giải pháp đã áp dụng
Thiết bị NAT xoá ánh xạ cổng khi kết nối không có lưu lượng; cần phát hiện client đã mất kết nối	KEEPALIVE mỗi 1 s giữ ánh xạ NAT; không nhận gói nào trong 10 s thì xem như client rời trận (đúng yêu cầu “mất kết nối quá 10 giây” của đề tài)
Keep-alive không được gửi trên kết nối đang có lưu lượng cao	Hẹn giờ theo thời điểm gửi keep-alive gần nhất, không theo thời điểm gửi gói bất kỳ; nếu không, kết nối gửi snapshot 20 Hz sẽ không bao giờ gửi keep-alive và thông tin điều khiển luồng không được cập nhật
Gói báo ngắt kết nối có thể bị mất	DISCONNECT gửi 3 lần liên tiếp, không cần ack; bên kia vẫn có cơ chế dự phòng timeout 10 s
Mạng nghẽn làm RTT tăng	Hai chế độ có trễ (hysteresis): SRTT > 250 ms chuyển sang BAD (snapshot 10 Hz, giảm chi tiết); trở về GOOD khi SRTT < 200 ms và đã ở BAD đủ 10 s; nếu vừa về GOOD chưa tới 10 s đã xấu lại thì thời gian chờ tăng lên 20 s để tránh dao động trạng thái
Rò rỉ tài nguyên khi đóng kết nối	Fail có tính lũy đẳng: trả mọi buffer của ReliabilityLayer, ChannelSet, FragmentAssembler; server trả connectionId về hàng đợi và gỡ ràng buộc playerId để người chơi kết nối lại được

4.7. Bộ giả lập mạng, ghi phát lại gói và đo hiệu năng

Bộ giả lập mạng. NetworkSimulator được đặt ngay dưới UdpPeer.Send: với mỗi datagram, bộ giả lập quyết định ngẫu nhiên việc hủy, nhân đôi hoặc giữ lại trong một khoảng trễ (có jitter, có thể trễ thêm để tạo đảo thứ tự), sau đó gửi ra socket khi tới thời điểm đến hạn trong Poll. Bộ sinh số ngẫu nhiên dùng hạt giống cố định nên cùng cấu hình tạo ra cùng chuỗi sự cố, giúp tái hiện lỗi một cách tất định. Bộ giả lập được kích hoạt qua biến môi trường IRONFRONT_SIM=typical, không cần biên dịch lại.

Hồ sơ	Trễ	Jitter	Mất gói	Đảo thứ tự	Trùng gói
lan	1 ms	0	0	0	0
good	30 ms	5 ms	0,5%	0	0
typical	50 ms	20 ms	5%	2%	0
bad	100 ms	50 ms	15%	5%	2%
awful	150 ms	100 ms	30%	10%	5%

Ghi và phát lại gói. Khi đặt IRONFRONT_PCAP=đường-dẫn, PacketLogger ghi mọi datagram vào tệp .ifpcap có đánh phiên bản (chiều, thời điểm, địa chỉ, byte thô). Công cụ Ironfront.Tools.PacketReplay đọc lại tệp, lọc theo khoảng thời gian và connectionId, giải mã header, ước lượng khoảng trống sequence và ghép cặp ack để tính RTT (Hình 3.11). Vì tệp có thể chứa joinTicket, tệp chỉ được lưu trong thư mục bị git bỏ qua.

Kết quả đo (localhost, .NET 8 Release, ghi trong plans/transport/reports/measurements.csv).

Phép đo	Kết quả
Phân tích header GSP	3,5 ns/lần, 0 byte cấp phát
16 kết nối, 30 Hz, 10 phút	16/16 kết nối ổn định, bộ đệm không phải cấp thêm
64 kết nối, 30 Hz, 20 s (đã khởi động)	64/64 kết nối; CPU 0,39% một nhân; Send và Poll cấp phát 0 byte
Ack bitfield bật / tắt	Một gói ack xác nhận 33 gói / 1 gói; còn 7 / 39 gói tin cậy chờ
Kiểm soát nghẽn	RTT cao: 20 → 10 Hz; thời gian ở BAD tối thiểu 20 s khi vừa chuyển trạng thái

4.8. Kiểm thử phân hệ

Bộ `Ironfront.Net.Transport.Tests` gồm 85 test xUnit, chạy trong CI trên Ubuntu và Windows:

Tệp test	Số test	Kịch bản tiêu biểu
<code>ControlAndSocketTests</code>	22	Gói điều khiển, timeout 10 s, DISCONNECT, socket thật trên localhost
<code>ReliabilityLayerTests</code>	19	Ack bitfield, sequence quay vòng, Karn, RTO, bỏ gói sau 10 lần
<code>HandshakeHardeningTests</code>	12	Cookie sai, cookie hết epoch, vé sai, vé trùng playerId, làm ngập yêu cầu
<code>ChannelAndFragmentTests</code>	11	Sai thứ tự, gói trùng, gói cũ trên kênh 1/3, ghép mảnh ngược thứ tự, giới hạn 8 nhóm
<code>ReliableChannelSurvivalTests</code>	8	Kênh tin cậy duy trì được qua giai đoạn mất gói kéo dài của bộ giả lập
<code>UdpPeerTests</code>	8	Ngân sách 1024 gói/Poll, bộ đệm socket, gói header sai
<code>PacketLoggerTests, Diagnostics</code>	5	Ghi – đọc lại .ifpcap, định dạng overlay chẩn đoán